

Semantic Runtime Specifications

An Abstract Machine and Execution Architecture for a Declarative,
Evidence-Bearing, Completion-Directed Language

Kundai Farai Sachikonye
Technical University of Munich
`kundai.sachikonye@wzw.tum.de`

June 6, 2026

Abstract

We specify the runtime of a declarative language whose programs do not script a sequence of steps but *declare a completion condition* and accumulate graded evidence toward it. Such a language poses runtime requirements that a conventional evaluator does not meet: a sub-computation may be resolved by any of several substrates (a deterministic primitive, an external tool, an oracle), each carrying a confidence; the result a program seeks is specified *backward*, as a target rather than a procedure; and every value that enters a conclusion must be *grounded* in an observation rather than freely generated. We give a complete, self-contained specification in five parts. (i) A compact recapitulation of the object language—its values, a confidence algebra on the unit interval, points and their resolution, propositions and their argumentation semantics, and the iteration constructs—so that the document stands alone. (ii) An *abstract machine* in the explicit-control–environment–store–continuation style, and a proof that it is *adequate*: it computes exactly the language’s big-step semantics. (iii) A *resolver/provider architecture*: typed interfaces that bind underspecified sub-computations to concrete substrates, with a *grounding* invariant that no value is committed to a conclusion without a matching observation. (iv) A *completion-directed* evaluation mechanism: tasks are targets in a refinement hierarchy embedded in a metric space, reached by backward navigation through *virtual sub-states*; we prove the logarithmic navigation bound and that the speed-up is impossible without virtual sub-states. (v) A best-first *scheduler* with a cascade of resolvers whose end-to-end accuracy obeys a multiplicative law, together with a content-addressed store and an observational decision log and state surface. The metatheory establishes adequacy, machine safety, scheduler fairness and termination, the cascade-accuracy bound and its saturation criterion, preservation of the grounding invariant, and conservation of confidence within the unit interval. The specification is precise enough to serve as an implementation blueprint and is related throughout to the standard literature on abstract machines, operational semantics, search, information geometry, and probabilistic computation.

Keywords: abstract machine, operational semantics, adequacy, completion-directed evaluation, backward navigation, virtual sub-states, resolver, scheduler, cascade, content-addressed store, grounding.

Contents

1	Introduction	2
1.1	What an evidence-bearing, completion-directed runtime must provide	2
1.2	Contributions	3
2	The Object Language, Recapitulated	3
2.1	Core syntax	3
2.2	Semantic domains	3
2.3	Big-step semantics (the specification to be realised)	4

3	The Abstract Machine	4
3.1	Configurations	4
3.2	Transitions	5
3.3	Adequacy	5
4	Resolvers and Providers	6
5	Completion-Directed Evaluation	6
5.1	The refinement hierarchy and its embedding	6
5.2	Virtual sub-states and backward navigation	7
5.3	Grounding meets completion	7
6	Scheduling, Cascades, and Determinism	7
6.1	Best-first scheduling	8
6.2	Resolver cascades and the accuracy law	8
6.3	Concurrency and determinism	8
7	The Store and the Project State	9
7.1	Content-addressed store	9
7.2	Decision log and state surface are observational	9
8	Metatheory	9
9	A Worked Execution	10
10	Related Work	11
11	Conclusion	11

1 Introduction

1.1 What an evidence-bearing, completion-directed runtime must provide

A runtime is a precise account of how a program is executed: a machine that turns syntax into results, together with the services—memory, scheduling, dispatch—that the machine needs. For a conventional language the account is well understood: an abstract machine in the tradition of Felleisen and Hieb [9], Landin [16] realises the language’s operational semantics [13, 20, 24], and adequacy theorems [1, 21] certify that the machine computes what the semantics prescribes. The language whose runtime we specify here is not conventional in three respects, each of which imposes a requirement that the classical account does not address.

- Plural substrates with confidence.** A sub-computation—resolving an ambiguous datum, scoring a hypothesis, answering a sub-query—may be discharged by any of several *substrates*: a deterministic primitive, an external tool, or an oracle whose answer is uncertain. Each returns not just a value but a *confidence*. The runtime must dispatch uniformly to these substrates and carry the confidence through.
- Backward specification.** A program declares *what completion looks like*—a target condition—rather than the steps to reach it. The runtime must compute the trajectory to the target, and do so efficiently. We show this is possible in logarithmic rather than linear work, but only through a device, the *virtual sub-state*, that the runtime must support.
- Grounding.** A conclusion must rest on observation. A value produced by an oracle and never checked against the world is *ungrounded*; the runtime must forbid such values from entering a committed result. This is a liveness/safety requirement on the scheduler, not on the semantics.

1.2 Contributions

We specify a runtime meeting these requirements and prove its guarantees.

- A self-contained recapitulation of the object language (section 2), so the document needs no companion.
- An abstract machine (section 3) and an *adequacy* theorem: the machine’s reflexive-transitive step relation computes exactly the big-step semantics (theorem 3.3).
- A resolver/provider architecture (section 4) with typed interfaces and a *grounding* invariant (theorem 4.4).
- Completion-directed evaluation by backward navigation through virtual sub-states (section 5), with the logarithmic-navigation theorem (theorem 5.5) and the *necessity* of virtual sub-states (theorem 5.6).
- A best-first scheduler with a resolver cascade (section 6), the multiplicative accuracy law (theorem 6.5), and a content-addressed store and observational project state (section 7).
- The metatheory (section 8): adequacy, machine safety, fairness and termination, cascade accuracy and saturation, grounding preservation, and confidence conservation.

Section 9 gives a worked execution; section 10 situates the design; section 11 concludes.

2 The Object Language, Recapitulated

We fix a core calculus—the fragment the runtime must execute—together with its semantic domains and big-step semantics. The presentation is compact but complete: nothing below depends on external material beyond the cited standard results.

2.1 Core syntax

Definition 2.1 (Core terms). Expressions e and statements s are

$$\begin{aligned}
 e &::= n \mid str \mid b \mid x \mid e \mathit{op} e \mid e(\bar{e}) \mid e[e] \mid \mathit{point}\{\bar{\ell} : e\} \mid \mathit{resolve}(e, \varsigma) \\
 s &::= \mathit{item} \ x = e \mid x := e \mid \mathit{seq}(\bar{s}) \mid \mathit{given}(e, s, s) \mid \mathit{foreach}(x, e, s) \mid \mathit{return}(e) \mid \mathit{ensure}(e) \\
 &\quad \mid \mathit{fun}(f, \bar{x}, s) \mid \mathit{prop}(P, \bar{M}, s) \mid \mathit{support}(M, e) \mid \mathit{contradict}(M, e) \\
 &\quad \mid \mathit{cycle}(x, e, s) \mid \mathit{drift}(x, e, s) \mid \mathit{roll}(s),
 \end{aligned}$$

with op ranging over the arithmetic, comparison, and logical operators, ς over resolution strategies (theorem 2.4), and x, f, P, M, ℓ over identifiers and labels. This is the fragment exercised by the runtime; the remaining surface constructs reduce to it.

2.2 Semantic domains

Definition 2.2 (Values). The value set $\mathbb{V}$ is generated by $v ::= \mathit{num}(r) \mid \mathit{str}(s) \mid \mathit{bool}(b) \mid \mathit{none} \mid \mathit{list}(\bar{v}) \mid \mathit{clo}(\bar{x}, s, \rho) \mid \mathit{pt}(p) \mid \mathit{ent}(p) \mid \mathit{floor}(F) \mid \mathit{corpus}(s)$, where a *point* $p = (\mathit{content}, \mathit{conf} \in [0, 1], (\overline{m_i, p_i}))$ carries content, a confidence, and a sub-stochastic list of interpretations ($\sum_i p_i \leq 1$); an *entity* is a resolved point; a *floor* F is a finite map from keys to weighted points; a *corpus* wraps a body of text.

Definition 2.3 (Confidence algebra). Confidences combine by the *noisy-or* on $[0, 1]$,

$$c \oplus d = 1 - (1 - c)(1 - d),$$

which is associative, commutative, monotone, has unit 0, and is bounded above by 1 (with 1 absorbing). It is the probability that at least one of two independent sources of confidence c, d succeeds.

Definition 2.4 (Resolution strategy). A *resolution strategy* ς maps a point p with normalised interpretations $(\overline{m_i}, \hat{p}_i)$ to a determinate outcome: the maximum-likelihood strategy returns the dominant interpretation when it clears an acceptance threshold θ and otherwise an uncertain outcome carrying the distribution and residual ambiguity $1 - A(p)$, where $A(p) = H(p)/\log_2 k$ is the normalised interpretation entropy; other strategies (Bayesian, conservative, exploratory, full) re-weight or re-select accordingly. Resolving collapses $\text{pt}(p)$ to $\text{ent}(\hat{p})$.

Definition 2.5 (Environment, store, debate, system state). An *environment* $\rho: \text{Var} \rightarrow \text{Loc}$ maps variables to locations; a *store* $\mu: \text{Loc} \rightarrow \mathbb{V}$ maps locations to values (mutation updates μ). A *debate state* $\Delta: \text{Motion} \rightarrow (\mathcal{A} \times \mathcal{C})$ maps each motion to multisets of weighted affirmations and contentions. A *system state* $\Psi = (\mathcal{H}, \mathcal{F})$ pairs an append-only *decision log* $\mathcal{H}$ with a *state surface* $\mathcal{F}$.

2.3 Big-step semantics (the specification to be realised)

The semantics has expression judgements $\langle e, \rho, \mu \rangle \Downarrow \langle v, \mu' \rangle$ and statement judgements $\langle s, \rho, \mu, \Delta \rangle \Downarrow \langle \chi, \mu', \Delta' \rangle$ with completion $\chi \in \{\text{nrm}\} \cup \{\text{ret } v\}$. We list the rules the runtime must honour; they are standard call-by-value [19, 24] except for the three distinctive forms.

$$\frac{\langle e, \rho, \mu \rangle \Downarrow \langle \text{pt}(p), \mu' \rangle \quad \hat{p} = \varsigma(p)}{\langle \text{resolve}(e, \varsigma), \rho, \mu \rangle \Downarrow \langle \text{ent}(\hat{p}), \mu' \rangle} \text{ (B-RESOLVE)}$$

$$\frac{\Delta_0 = [\overline{M \mapsto (\emptyset, \emptyset)}] \quad \langle s, \rho, \mu, \Delta_0 \rangle \Downarrow \langle _, \mu', \Delta_1 \rangle}{\langle \text{prop}(P, \overline{M}, s), \rho, \mu, \Delta \rangle \Downarrow \langle \text{nrm}, \mu' [P \mapsto \text{verdict}_\varsigma(\Delta_1)], \Delta \rangle} \text{ (B-PROP)}$$

$$[8pt] \frac{\langle e, \rho, \mu \rangle \Downarrow \langle \text{num}(c), \mu' \rangle \quad c' = \text{clip}[0, 1](c)}{\langle \text{support}(M, e), \rho, \mu, \Delta \rangle \Downarrow \langle \text{nrm}, \mu', \Delta [M \mapsto \text{A}(c', c', 1)] \rangle} \text{ (B-SUPPORT)}$$

where verdict_ς scores each motion (an affirmations-minus- contentions aggregate under strategy ς , clipped to $[0, 1]$) and labels it Supported/Contradicted/Inconclusive by thresholds. The iteration forms *cycle*/*drift*/*roll* run their body once per floor entry, per sampled corpus fragment, and until a convergence predicate fires, respectively. We take this semantics as given and specify a machine that realises it.

3 The Abstract Machine

We give a small-step abstract machine in the control–environment–store– continuation style [1, 9]. The machine makes evaluation order, the store, and the pending context explicit, and exposes exactly the points at which the runtime dispatches to resolvers and providers (section 4).

3.1 Configurations

Definition 3.1 (Machine state). A *machine state* is a tuple

$$\varsigma = \langle c, \rho, \mu, \kappa, \Delta, \Psi \rangle,$$

where c is the *control* (a term being evaluated, or a value being returned to a continuation), ρ the environment, μ the store, κ the *continuation* (the pending context), Δ the debate state, and Ψ the system state. Continuations are

$$\kappa ::= \text{halt} \mid \text{arg}(\bar{e}, \rho, \kappa) \mid \text{op}(op, v, \kappa) \mid \text{seq}(s, \rho, \kappa) \mid \text{bind}(x, \kappa) \mid \text{res}(\varsigma, \kappa) \mid \dots$$

each frame recording one suspended elimination form. A state is *final* when c is a value v and $\kappa = \text{halt}$.

3.2 Transitions

The step relation $\varsigma \longrightarrow \varsigma'$ decomposes evaluation into atomic steps. We give the representative rules; the rest follow the same discipline.

$$\begin{aligned}
\langle e_1 \text{ op } e_2, \rho, \mu, \kappa, \Delta, \Psi \rangle &\longrightarrow \langle e_1, \rho, \mu, \text{op}_1(\text{op}, e_2, \rho, \kappa), \Delta, \Psi \rangle \\
\langle v_1, \rho', \mu, \text{op}_1(\text{op}, e_2, \rho, \kappa), \Delta, \Psi \rangle &\longrightarrow \langle e_2, \rho, \mu, \text{op}_2(\text{op}, v_1, \kappa), \Delta, \Psi \rangle \\
\langle v_2, \rho, \mu, \text{op}_2(\text{op}, v_1, \kappa), \Delta, \Psi \rangle &\longrightarrow \langle \llbracket \text{op} \rrbracket(v_1, v_2), \rho, \mu, \kappa, \Delta, \Psi \rangle \\
\langle x, \rho, \mu, \kappa, \Delta, \Psi \rangle &\longrightarrow \langle \mu(\rho(x)), \rho, \mu, \kappa, \Delta, \Psi \rangle \\
\langle e(\bar{e}), \rho, \mu, \kappa, \Delta, \Psi \rangle &\longrightarrow \langle e, \rho, \mu, \text{arg}(\bar{e}, \rho, \kappa), \Delta, \Psi \rangle \\
\langle \text{clo}(\bar{x}, s, \rho_0), \dots, \text{arg}(\bar{v}, _, \kappa), \dots \rangle &\longrightarrow \langle s, \rho_0[\bar{x} \mapsto \bar{\ell}], \mu[\bar{\ell} \mapsto v], \kappa, \Delta, \Psi \rangle
\end{aligned}$$

For the distinctive forms, the machine *suspends to a resolver*: evaluating $\text{resolve}(e, \varsigma)$ pushes $\text{res}(\varsigma, \kappa)$, and when the operand reduces to a point the machine emits a *resolution request* (section 4) whose answer re-enters as the entity value; evaluating $\text{support}(M, e)$ updates Δ once e is a number; evaluating prop installs a fresh inner Δ_0 , runs the body, and writes the verdict to the store and an entry to $\Psi.\mathcal{H}$.

Definition 3.2 (Evaluation by the machine). Let $\longrightarrow^*$ be the reflexive-transitive closure of $\longrightarrow$. The machine *evaluates* a closed term t to v if $\langle t, \emptyset, \mu_0, \text{halt}, \Delta_0, \Psi_0 \rangle \longrightarrow^* \langle v, _, \mu', \text{halt}, \Delta', \Psi' \rangle$.

3.3 Adequacy

Theorem 3.3 (Adequacy). *For every closed expression e and store μ : the machine evaluates e to v with final store μ' iff $\langle e, \emptyset, \mu \rangle \Downarrow \langle v, \mu' \rangle$ in the big-step semantics; and likewise for statements, with the machine reaching a final state carrying completion χ , store μ' , and debate Δ' iff $\langle s, \emptyset, \mu, \Delta \rangle \Downarrow \langle \chi, \mu', \Delta' \rangle$. The machine is therefore a correct and complete realisation of the semantics.*

Proof sketch. This is the standard correspondence between a big-step semantics and a CEK/CESK machine [1, 9, 21]. ($\Leftarrow$, soundness of the machine) By induction on the big-step derivation, exhibiting for each rule a finite block of machine steps that pushes the appropriate continuation frames, evaluates sub-terms (using the IH), and pops the frame with the combined value; the store and debate threads are passed unchanged where the rule does not touch them and updated exactly where it does (e.g. B-SUPPORT updates Δ , B-PROP installs and discharges Δ_0). ($\Rightarrow$, completeness) By induction on the length of $\longrightarrow^*$, using a *decomposition lemma*: every non-final state uniquely decomposes into a redex in an evaluation context (the continuation), so each maximal run corresponds to a unique big-step derivation; the resolution-request steps correspond to the premises of B-RESOLVE with the resolver supplying the $\varsigma(p)$ side condition. Determinacy of $\longrightarrow$ (each state has at most one successor) makes the correspondence a bijection on terminating runs. $\square$

Corollary 3.4 (Machine safety). *On the statically typed fragment (no dynamic type), a well-typed initial state never reaches a stuck non-final configuration: every non-final state has a successor, except a deliberate abort from a failed ensure (a defined outcome in the sense of Hoare [11]) or a state awaiting a resolver answer.*

Proof. By adequacy (theorem 3.3) the machine's reachable states are exactly the sub-derivations of a big-step evaluation, and progress for the big-step semantics on the static fragment [19, 25] transfers: each non-value control has a redex with a defined transition, the only non-progressing outcomes being the defined abort and the resolver wait. $\square$

4 Resolvers and Providers

The machine reduces a program to a sequence of *requests*: primitive operations to be computed, and underspecified sub-computations to be resolved. We give the two interfaces that service them and the invariant that disciplines their use.

Definition 4.1 (Provider). A *provider* for an operation name op with signature $\bar{\tau} \rightarrow \tau'$ is a (partial) function $P_{op}: \mathbb{V}^* \rightarrow \mathbb{V}$ that computes op deterministically on values of the declared types. A *registry* Reg is a finite map from operation names to providers. The machine’s op step consults Reg : $\llbracket op \rrbracket = \text{Reg}(op)$.

Definition 4.2 (Resolver). A *resolver* is a map

$$R: \text{Request} \times \text{Context} \longrightarrow \mathbb{V} \times [0, 1],$$

that answers an underspecified request (resolve a point, score a motion, satisfy a sub-target) with a value and a confidence, given a context (the ambient store, relevant evidence, and the active strategy). A resolver is classified by its substrate: *deterministic* (a pure function, e.g. applying a strategy to a point’s interpretation distribution), *tool* (an external computation that reads the world), or *oracle* (a substrate whose answer is uncertain). Each resolver carries a *catalytic power* $\kappa(R) \in [0, 1]$ (section 6).

Definition 4.3 (Requests and observations). A *request* is *generative* if it proposes a value (an oracle answer, a candidate completion) and *observational* if it reads the world (a tool call, a measurement, a database lookup). A value v produced by a generative request is *grounded* once an observational request has confirmed it within tolerance; otherwise it is *ungrounded*.

Invariant 4.4 (Grounding). No ungrounded value is committed to a conclusion. Operationally: a generative resolution that contributes to a proposition verdict, a returned result, or a published state must be matched, before commitment, by an observational resolution that confirms it; the scheduler (section 6) holds the generative value in a pending state until the matching observation completes, and discards it on failure.

Remark 4.5 (Closure: action requires return). theorem 4.4 is the runtime’s closure discipline: every outbound (generative) step must be closed by an inbound (observational) step before its effect is allowed to stand. Ungrounded generation—an oracle answer that never faces the world—is precisely what the invariant forbids, and it is the structural account of, and guard against, unfounded conclusions. The invariant constrains the *scheduler*, not the semantics: a program’s meaning is unchanged, but the runtime refuses to realise meanings built on uncorroborated values.

5 Completion-Directed Evaluation

A task declares a *completion condition*—a target Γ_f —rather than a procedure. The runtime computes a trajectory that reaches Γ_f . We show this is done in logarithmic work by *backward* navigation through a refinement hierarchy embedded in a metric space, and that the speed-up depends essentially on *virtual sub-states*.

5.1 The refinement hierarchy and its embedding

Definition 5.1 (Ternary refinement hierarchy). A *ternary refinement hierarchy* of depth k is a sequence of partitions $P_0, \dots, P_k$ of a target space with $|P_0| = 1$, each block of P_j refining into exactly three children of P_{j+1} ; the leaves P_k number $N = 3^k$. A *target* is a leaf; a *trajectory* is a root-to-leaf path.

Definition 5.2 (Metric embedding). An *embedding* embed maps each node to a point of the cube $[0, 1]^3$ by the base-3 expansion of its address along three coordinates, and equips $[0, 1]^3$ with a product Fisher–Rao metric $\mathbf{d}$ [2, 5]. Under embed, the parent of a node is obtained by deleting the last base-3 digit of its coordinate—an $O(1)$ operation—and the ball of radius r about a leaf contains $O(r^3 3^k)$ leaves.

5.2 Virtual sub-states and backward navigation

Definition 5.3 (Virtual sub-state). A *sub-state decomposition* of a coordinate $s \in [0, 1]$ is a triple $(s_1, s_2, s_3) \in \mathbb{R}^3$ with mean $\frac{1}{3}(s_1 + s_2 + s_3) = s$. It is *virtual* if some $s_i \notin [0, 1]$ (a locally infeasible component whose aggregate is feasible), and *physical* otherwise.

Definition 5.4 (Backward navigation). Given a target leaf Γ_f , backward navigation returns the trajectory $\Gamma_f = p_k, p_{k-1}, \dots, p_0$ by iterating the $O(1)$ parent map of theorem 5.2, each step admitting a (possibly virtual) sub-state decomposition of the coordinate.

Theorem 5.5 (Logarithmic navigation). *Backward navigation from any target leaf to the root of a depth- k ternary refinement hierarchy visits exactly $k = \log_3 N$ nodes, each in $O(1)$ work under the embedding; total cost $O(\log_3 N)$.*

Proof. Each step deletes one base-3 digit (the $O(1)$ parent map, theorem 5.2), moving up one level; from depth k to 0 takes k steps; $k = \log_3 N$. $\square$

Theorem 5.6 (Necessity of virtual sub-states). *Backward navigation restricted to physical sub-state decompositions has worst-case cost $\Theta(N)$ —no better than forward enumeration. The logarithmic bound of theorem 5.5 requires virtual sub-states.*

Proof sketch. The geodesic short-cut in the continuous embedding requires evaluating the metric at a resolution finer than the cell width of the next level, which is attainable only through decompositions ranging over $\mathbb{R}^3$ rather than the unit cube. Confined to physical decompositions, the navigator must instead distinguish a node’s child among the candidates at each level by direct comparison; summing the candidate counts $\sum_{j=0}^k 3^j = \Theta(N)$. Thus the freedom to pass through locally infeasible (virtual) intermediate coordinates is exactly what buys the exponential reduction; a forward construction, which must realise each intermediate as a physical block, cannot use it. $\square$

Corollary 5.7 (Observational forward, analytical backward). *Forward construction realises each step physically and is $\Theta(N)$; backward completion computes intermediates analytically and is $O(\log_3 N)$. A task is therefore best executed by declaring its target and navigating backward, which is the runtime’s default for goal-resolution requests.*

5.3 Grounding meets completion

Backward navigation produces a *candidate* trajectory; by theorem 4.4 the leaf value it proposes is generative and must be observed before commitment. Completion-directed evaluation thus interleaves an analytical backward pass (cheap, generative) with an observational forward check (grounding), and commits only when both agree within tolerance.

6 Scheduling, Cascades, and Determinism

The machine emits requests; the scheduler decides their order, dispatches them to resolvers/providers, composes resolvers into cascades, and enforces grounding.

6.1 Best-first scheduling

Definition 6.1 (Work item and priority). A *work item* is a pending request together with its current state (coordinate) s and its target Γ_f . Its *priority* is $\pi = -d(s, \Gamma_f)$: items closer to their target are scheduled first. The scheduler maintains a priority queue of enabled items and repeatedly dispatches the highest-priority one—best-first search in the sense of Dijkstra [6], Hart et al. [10].

Definition 6.2 (Fair policy). A scheduling policy is *fair* if every continuously enabled work item is eventually dispatched. We require the scheduler to be fair, e.g. by ageing priorities so that no enabled item is starved [7].

Theorem 6.3 (Liveness under fairness). *Under a fair policy, every enabled resolution is eventually performed; hence if a computation’s request DAG is finite and acyclic, the computation terminates, and if every leaf request has a resolver, it terminates with all requests discharged.*

Proof. Fairness gives that no enabled item waits forever (theorem 6.2); on a finite acyclic DAG the dependency order is a well-founded partial order, so by induction over it every item becomes enabled and is then, by fairness, eventually dispatched; the process drains the DAG in finite time. The qualifier on leaf resolvers ensures no item blocks for want of a substrate. $\square$

6.2 Resolver cascades and the accuracy law

A request is often discharged by a *cascade*: a sequence of resolvers each refining the previous answer. We characterise the end-to-end accuracy.

Definition 6.4 (Catalytic power and cascade). The *catalytic power* $\kappa(R) \in [0, 1]$ of a resolver is the fraction by which it reduces the residual distance of an answer to the truth: applying R to an answer at residual δ leaves residual $(1 - \kappa)\delta$. A *cascade* $\Gamma = R_1 \diamond \dots \diamond R_n$ applies the resolvers in sequence.

Theorem 6.5 (Multiplicative accuracy law). *The catalytic power of a cascade is*

$$\kappa(R_1 \diamond \dots \diamond R_n) = 1 - \prod_{i=1}^n (1 - \kappa(R_i)),$$

the noisy-or aggregate (theorem 2.3) of the stage powers; equivalently the residual multiplies, $\delta_n = \delta_0 \prod_i (1 - \kappa_i)$.

Proof. By induction on n . One stage: residual $\delta_0 \mapsto (1 - \kappa_1)\delta_0$. If after $n - 1$ stages the residual is $\delta_0 \prod_{i < n} (1 - \kappa_i)$, the n -th leaves $\delta_0 \prod_{i \leq n} (1 - \kappa_i)$; the cascade power is $1 - \delta_n/\delta_0 = 1 - \prod_i (1 - \kappa_i)$, which is the $\oplus$ -fold of the κ_i . $\square$

Corollary 6.6 (Saturation). *A cascade drives the residual to 0 (power to 1) iff $\sum_i \kappa(R_i) = \infty$. A cascade of fixed per-stage power $\kappa > 0$ saturates geometrically; one with $\kappa_i = 2^{-i}$ does not.*

Proof. $\log \prod_i (1 - \kappa_i) = \sum_i \log(1 - \kappa_i)$ diverges to $-\infty$ iff $\sum_i \kappa_i = \infty$ (since $\log(1 - x) \sim -x$), i.e. residual $\rightarrow 0$; this is the Borel–Cantelli dichotomy. $\square$

6.3 Concurrency and determinism

Proposition 6.7 (Determinism relative to the oracle assignment). *Fix an assignment $\mathcal{O}$ of answers to every oracle request. Then the runtime is deterministic: the machine step relation is a partial function (theorem 3.3), provider answers are functions of their arguments (theorem 4.1), and oracle answers are fixed by $\mathcal{O}$. Two independent work items (neither’s target nor inputs depending on the other’s output) may be dispatched concurrently without affecting the result, because their effects on disjoint store locations and distinct log entries commute.*

Proof. Determinism: each source of choice is resolved—transitions by determinacy of $\longrightarrow$, providers by functionality, oracles by $\mathcal{O}$. Commutation: independent items touch disjoint store locations (no read-write or write-write hazard) and append to the log, an operation that is conflict-free up to the recorded order [15]; hence their two interleavings yield the same final store and the same multiset of log entries. $\square$

7 The Store and the Project State

7.1 Content-addressed store

Definition 7.1 (Content-addressed store). The runtime’s long-term store is *content-addressed*: the location of a value is computed from the value, $\ell = \text{embed}(\text{content})$, rather than assigned independently. Retrieval is by coordinate proximity—the nearest stored coordinate under $\mathbf{d}$ —and storage and retrieval share one operation. There is no separate key space: an address is never supplied, only derived.

Proposition 7.2 (Storage–retrieval coincidence). *Under content addressing, $\text{retrieve}(\text{embed}(x)) = x$ for any stored x whose nearest neighbour is itself; storing x and later retrieving by its content recover x exactly, and approximate content retrieves the nearest stored item.*

Proof. embed places x at $\text{embed}(x)$; retrieval returns $\arg \min_y \mathbf{d}(\text{embed}(x), \text{embed}(y))$, which is x when x is stored and otherwise the nearest stored coordinate. The embedding’s injectivity on stored contents makes exact recovery exact. $\square$

7.2 Decision log and state surface are observational

Definition 7.3 (Project threading). Two effects accompany evaluation beyond the store: a proposition verdict or a resolution *appends* a record to the decision log $\Psi.\mathcal{H}$, and a publish *updates* the state surface $\Psi.\mathcal{F}$. Both are write-only with respect to the core: no machine transition reads $\mathcal{H}$ or $\mathcal{F}$.

Theorem 7.4 (Observational separation). *Erasing $\mathcal{H}$ and $\mathcal{F}$ from every machine state yields exactly the runtime over the bare core (store and debate only): the projection that drops Ψ commutes with $\longrightarrow$. Hence the log and surface record the computation but do not steer it; the result is independent of them.*

Proof. The only transitions touching Ψ append to $\mathcal{H}$ or update $\mathcal{F}$ (theorem 7.3); none reads Ψ . Therefore the projection $\langle c, \rho, \mu, \kappa, \Delta, \Psi \rangle \mapsto \langle c, \rho, \mu, \kappa, \Delta \rangle$ maps each transition to the corresponding core transition and is a simulation in both directions; the diagram commutes, and the final control, store, and debate are unchanged by erasure. $\square$

8 Metatheory

We collect the runtime’s guarantees; several restate results proved above, here assembled as the runtime’s contract.

Theorem 8.1 (Runtime contract). *The runtime satisfies:*

- (i) **Adequacy** (theorem 3.3): *it computes exactly the object language’s big-step semantics.*
- (ii) **Safety** (theorem 3.4): *on the static fragment no run gets stuck except at a defined abort or a resolver wait.*
- (iii) **Liveness** (theorem 6.3): *under fair scheduling every enabled resolution is performed; finite acyclic request DAGs terminate.*
- (iv) **Efficiency** (theorem 5.5): *goal resolution by backward navigation is $O(\log_3 N)$, optimal up to the necessity of virtual sub-states (theorem 5.6).*

- (v) **Accuracy** (theorem 6.5, theorem 6.6): a resolver cascade has end-to-end power $1 - \prod_i (1 - \kappa_i)$ and saturates iff $\sum_i \kappa_i = \infty$.
- (vi) **Grounding** (theorem 4.4): no ungrounded value is committed to a conclusion.
- (vii) **Confidence conservation**: every confidence the runtime computes lies in $[0, 1]$.
- (viii) **Observational state** (theorem 7.4): the decision log and state surface record but do not alter the computation.

Proof. (i)–(vi), (viii) are the cited theorems. (vii): confidences arise from resolver answers (in $[0, 1]$ by theorem 4.2), from clipping in B-SUPPORT, from strategy outputs (in $[0, 1]$ by theorem 2.4), and from cascade aggregation by $\oplus$, which is bounded above by 1 and below by 0 (theorem 2.3); all constructors preserve $[0, 1]$, so by induction over the evaluation every computed confidence is in $[0, 1]$. $\square$

Theorem 8.2 (Grounding preservation). *If the scheduler holds every generative value pending until a matching observational resolution succeeds, then theorem 4.4 holds of every reachable state: no reachable committed conclusion depends on an ungrounded value.*

Proof. By induction on the run. Initially no value is committed. The only steps that commit a value to a conclusion are: writing a proposition verdict (B-PROP), returning a result, and publishing state. By hypothesis the scheduler does not enable these on a value still pending grounding; thus at each such step the contributing value has a completed matching observation, preserving the invariant. Generative values whose observation fails are discarded and never reach a committing step. $\square$

Remark 8.3 (Termination is conditional). As for any language with general recursion and unbounded iteration [12], termination of while/roll is undecidable; theorem 6.3 gives termination for finite acyclic request DAGs and bounded iteration. The runtime therefore guarantees progress and fairness unconditionally and termination under the stated finiteness condition, exactly as the object language guarantees termination only for its bounded constructs.

9 A Worked Execution

Consider resolving an ambiguous datum inside a proposition, with a two-stage resolver cascade and the grounding check.

1. The machine evaluates $\text{prop}(P, \overline{M}, s)$ (B-PROP): installs Δ_0 , runs the body.
2. The body reaches item $d = \text{resolve}(\text{point}\{\dots\}, \varsigma)$. The machine builds $\text{pt}(p)$ and pushes $\text{res}(\varsigma, \kappa)$, emitting a *resolution request* for p .
3. The scheduler dispatches the request to a cascade $R_1 \diamond R_2$: R_1 a deterministic strategy resolver ($\kappa_1 = 0.6$) producing a candidate entity; R_2 an oracle ($\kappa_2 = 0.5$) refining it. By theorem 6.5 the cascade power is $1 - (0.4)(0.5) = 0.8$.
4. The oracle answer is *generative*; by theorem 4.4 it is held pending. The scheduler issues an *observational* request (a tool lookup); on agreement within tolerance the entity is grounded and committed as d , with confidence the cascade aggregate; on disagreement it is discarded and the next candidate sought.
5. Back in the proposition body, given $\phi : \text{support}(M, d.\text{conf})$ updates Δ_0 (B-SUPPORT). After the body, $\text{verdict}_\varsigma(\Delta_1)$ is written to the store and appended to the decision log $\Psi.\mathcal{H}$ (theorem 7.3).

By theorem 3.3 the value computed equals the big-step result; by theorem 7.4 the log entry does not change it; by theorem 8.2 the committed verdict rests only on grounded values.

10 Related Work

Abstract machines and adequacy. The machine is in the SECD/CEK/CESK lineage [9, 16]; the adequacy proof follows the functional correspondence between evaluators and machines [1, 21] and the standard relationship to big-step and structural operational semantics [13, 20, 24]. Type safety transfers by the syntactic method [19, 25].

Dispatch and effects. The resolver/provider split is a typed dispatch interface in the spirit of effect handlers and the monadic account of computational effects [18, 23]; the registry is a small module system [4].

Search and navigation. Best-first scheduling by distance-to-target is Dijkstra/A* search [6, 10]; the backward-navigation bound and the metric embedding draw on information geometry [2, 5] and a fixed-point view of convergent refinement [3]. The content-addressed store realises storage and retrieval as one coordinate operation.

Uncertainty. Confidences and their noisy-or aggregation place the runtime in the probabilistic-computation tradition [14, 17] and the theory of evidence [22]; proposition verdicts realise a quantitative argumentation [8].

Concurrency. Fairness and the conflict-free composition of independent work items follow classical concurrency [7, 15].

The novelty is the assembly: an adequate machine whose request stream is serviced by substrate-agnostic resolvers under a grounding invariant, with goal resolution by backward navigation through virtual sub-states, a cascade-accuracy law, and an observational project state—a runtime tailored to a language that declares targets and reasons under evidence.

11 Conclusion

We have specified the runtime of a declarative, evidence-bearing, completion-directed language and proved its contract. An abstract machine realises the language’s semantics adequately and safely; a resolver/provider architecture binds underspecified sub-computations to deterministic, tool, and oracle substrates while a grounding invariant forbids any ungrounded value from entering a conclusion; goal resolution proceeds by backward navigation through a metric-embedded refinement hierarchy in logarithmic work, provably impossible without virtual sub-states; a fair best-first scheduler composes resolvers into cascades whose accuracy follows a multiplicative law with a sharp saturation criterion; and a content-addressed store with an observational decision log and state surface completes the picture. The specification is self-contained and constructive: the machine fixes the evaluator, the interfaces fix the dispatch, the navigation fixes goal resolution, and the scheduler fixes the order and the grounding—each with a stated and proved guarantee, so that an implementation has both a blueprint and a contract to meet.

References

- [1] Mads Sig Ager, Dariusz Biernacki, Olivier Danvy, and Jan Midtgaard. A functional correspondence between evaluators and abstract machines. In *Proceedings of the 5th ACM SIGPLAN International Conference on Principles and Practice of Declarative Programming (PPDP)*, pages 8–19, 2003.
- [2] Shun-ichi Amari. *Information Geometry and Its Applications*. Springer, 2016.

- [3] Stefan Banach. Sur les opérations dans les ensembles abstraits et leur application aux Équations intégrales. *Fundamenta Mathematicae*, 3:133–181, 1922.
- [4] Luca Cardelli and Peter Wegner. On understanding types, data abstraction, and polymorphism. *ACM Computing Surveys*, 17(4):471–523, 1985.
- [5] Nikolai N. Čencov. *Statistical Decision Rules and Optimal Inference*, volume 53 of *Translations of Mathematical Monographs*. American Mathematical Society, 1982.
- [6] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.
- [7] Edsger W. Dijkstra. Cooperating sequential processes. In F. Genuys, editor, *Programming Languages: NATO Advanced Study Institute*, pages 43–112. Academic Press, 1968.
- [8] Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995.
- [9] Matthias Felleisen and Robert Hieb. The revised report on the syntactic theories of control and state. *Theoretical Computer Science*, 103(2):235–271, 1992.
- [10] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [11] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [12] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, 3 edition, 2006.
- [13] Gilles Kahn. Natural semantics. In *STACS 1987*, volume 247 of *Lecture Notes in Computer Science*, pages 22–39. Springer, 1987.
- [14] Dexter Kozen. Semantics of probabilistic programs. *Journal of Computer and System Sciences*, 22(3):328–350, 1981.
- [15] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
- [16] Peter J. Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 1964.
- [17] Annabelle McIver and Carroll Morgan. *Abstraction, Refinement and Proof for Probabilistic Systems*. Springer, 2005.
- [18] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.
- [19] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [20] Gordon D. Plotkin. A structural approach to operational semantics. *Technical Report DAIMI FN-19, Aarhus University; reprinted in Journal of Logic and Algebraic Programming 60–61 (2004) 17–139*, 1981.
- [21] John C. Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM Annual Conference*, pages 717–740, 1972.

- [22] Glenn Shafer. *A Mathematical Theory of Evidence*. Princeton University Press, 1976.
- [23] Philip Wadler. The essence of functional programming. In *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 1–14, 1992.
- [24] Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. MIT Press, 1993.
- [25] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.